

Software Requirements Specification

for

Connect-Kitchen

Prepared by Ashar Zamir, Bilal Ahmed, Anas
Saleem

**National University of computing and
emerging sciences**

Date Created: December 1, 2024

Table of Contents

Table of Contents

1. Introduction	1
1.1 Purpose	1
1.2 Document Conventions	1
1.3 Intended Audience and Reading Suggestions	1
1.3 Product Scope	2
1.4 References	2
2. Overall description	2
2.1 Product Perspective	2
2.2 Product Functions	2
2.3 User Classes and Characteristics	3
2.4 Operating Environment	3
2.5 User Documentation	4
2.6 Design and Implementation Constraints	4
2.7 Assumptions and dependencies	5
2.8 Process Model	5
2.8.1 Customer Process Flow	5
2.8.2 Admin Process Flow	6
2.9 Project Plan	6
2.9.1 Project Plan	6
3. External Interface Requirements	8
3.1 User Interface	8
3.2 Software Interfaces	9
4. System Features	10
4.1 Admin Features	10
4.2 User Features	10
5. Functional Requirements	10
5.1 Admin Features	10
5.2 User Features	11
6. Non-Functional Requirements	11

1. Introduction

1.1 Purpose

The Connect Kitchen project aims to create a centralized platform for chefs, home cooks, and food enthusiasts to share recipes, interact socially, and order ingredients online. This SRS defines the software requirements necessary to achieve the goals of the project.

1.2 Document Conventions

This document uses the following conventions:

- Features are categorized into Admin and User features.
- Lists are provided with bullet points or numbered lists where appropriate.
- Section headers indicate different components of the software system.

1.3 Intended Audience and Reading Suggestions

- Users: Registered users and administrators who interact with the application.
- Stakeholders: Business owners or project managers who need an overview of the features and functionalities.

1.4 Product Scope

- User-friendly interface for creating social media accounts , browsing and purchasing products.
- Product catalog management for small business owners.
- Basic order tracking for customers.

1.5 References

- Stack overflow.
- MDN Docs. <https://developer.mozilla.org/en-US/docs/Web/JavaScript>
- GitHub.
- font-awesome. <https://cdnjs.com/libraries/font-awesome>.

2. Overall description

This section provides a broad overview of the “Connect Kitchen”, highlighting its purpose, key features, and general context.

2.1 Product Perspective

Connect Kitchen combines the functionalities of social media platforms and recipe management systems, integrating user interactions with order placement.

- A user-friendly interface for customers to create posts, browse, search, and purchase products and social media functionalities.
- Administrative tools for managing discount vouchers, orders history and update category.
- A responsive design, making it accessible across different desktops.

2.2 Product Functions

The application includes the following core functionalities:

- **Customer Module:**
 - User registration and login.
 - Create a profile.
 - Product browsing, search, and filtering.
 - checkout process.
 - Like/comment/follow.
- **Admin Module:**
 - View and manage customer orders.
 - Analytics dashboard for sales tracking.
 - Issue vouchers.
 - Update Categories.
- **Additional Features:**
 - Secure payment processing.
 - Notifications for order status updates.

2.3 User Classes and Characteristics

The primary users of the application are divided into the following categories:

1. User:

- Casual and frequent shoppers.
- Require an intuitive interface for browsing and purchasing.
- Expect smooth socializing.

2. Administrators:

- Responsible for managing the order catalog, categories, and vouchers.
- Need an efficient dashboard for monitoring sales and system health.

2.4 Operating Environment

The platform will run on a web-based environment, compatible with modern browsers.

2.5 Design and Implementation Constraints

The application operates in the following environments:

• Frontend:

- Developed using React.js for web application.

• Backend:

- Built with Express.js and Node.js, connecting to a PostgreSQL database.

• Minimum System Requirements:

- A device with an internet connection.
- Browser with JavaScript enabled.

1- Technology Constraints

- The application must use React.js for the frontend and Node.js with Express.js for the backend.
- PostgreSQL is the chosen database; no other database technologies will be used.

2- Localization Constraints

- The application must support English as the default language, with optional support for additional languages in the future.

2.6 User Documentation

The application is accompanied by the following documentation:

- **User Guide:**
 - **Description:** A comprehensive guide that explains how to install, configure, and use the software. It should include detailed instructions for each feature like for customer account creation, browsing, and order placement.
 - **Delivery Format:** Typically delivered as a PDF, HTML, or within the software as a built-in help system.
- **Admin Manual:**
 - Instructions for managing product categories, monitoring orders, and generating reports.
- **API Documentation:**
 - Details for developers integrating third-party services or extending the platform.

2.7 Assumptions and dependencies

- **Assumptions:**
 - Customers have access to the internet and modern web browsers.
 - Admins are familiar with basic e-commerce operations.
- **Dependencies:**
 - Uses third-party libraries and frameworks, which need regular updates.

2.8 Process Model

The **Process Model** defines the flow of data and control through the system and outlines the key processes involved in the Connect-Kitchen platform. The process model is divided into two primary categories: **Customer-related processes** and **Admin-related processes**.

2.8.1 Customer Process Flow

- **User Registration and Login:**
 - The user provides their credentials, and the system validates them.
 - On successful login, the user gains access to their personalized dashboard.
- **Product Browsing and Selection:**
 - Users can filter products by categories or search by name.
 - Product details (price, description, and reviews) are displayed.
- **Add to Cart:**
 - Users can add products to their shopping cart.
 - Cart displays the total number of items and cost.
- **Checkout and Payment:**
 - The user reviews the cart, enters shipping information, and then check out.

2.8.2 Admin Process Flow

- **Product Management:**
 - Admin can remove products post.
 - The system updates the product catalog, reflecting changes in real-time.
- **Order Management:**
 - Admin can review the status of ordered items.
 - Order status updates (e.g., accepted, rejected and pending) are monitored.
- **Customer Management:**
 - Admin can view and manage customer profiles and order histories.
- **Reporting:**

- Admin views and generates sales reports from the dashboard.

2.9 Project Plan

The **Project Plan** provides an outline for the stages, tasks, and timeline for developing the MartCom e-commerce platform.

2.9.1 Project Plan

- **Phase 1: Planning and Design (Duration: 4 Weeks)**
 - **Task 1:** Requirement gathering and analysis.
 - **Task 2:** Creating initial wireframes and system architecture.
 - **Milestone 1:** SRS document and initial wireframes completed.
- **Phase 2: Frontend and Backend Development (Duration: 8 Weeks)**
 - **Task 1:** Develop customer-facing features (product browsing, cart, checkout, order , profile page).
 - **Task 2:** Develop admin-facing features (delete inappropriate comments, order order history, reporting, voucher issuance).
 - **Milestone 2:** Basic functional system prototype completed (customer and admin dashboards, product catalog).
- **Phase 3: Integration and Testing (Duration: 2 Weeks)**
 - **Task 1:** Integrate frontend with backend.
 - **Task 2:** database creation and integration for testing.
 - **Milestone 3:** Fully integrated system with successful tests.
- **Phase 4: Post-Launch Support (Ongoing)**
 - **Task 1:** Monitor system performance.
 - **Task 2:** Regular updates and feature enhancements.

2.10 Feasibility report

2.10.1 Technical Feasibility

- **Technology Stack:**
 - Frontend: React.js (modern, responsive UI/UX)
 - Backend: Express.js (robust API development) and Node.js
 - Database: PostgreSQL (reliable relational database)
 -
- **System Requirements:**
 - Development Tools: VS Code, Postman, Git
 - Deployment: Linux-based VPS or cloud infrastructure
 - User Devices: Compatible with browsers on desktops and laptops.

2.10.2 Operational Feasibility

- **Team Composition:**
 - **Development Team:** 3 developers (frontend, backend, full-stack).
 - **UI/UX Designer:** Focused on user interface and experience design.
 - **Quality Assurance (QA):** Ensuring application stability and usability.
 - **Customer Support:** Handling customer inquiries, returns, and complaints.
- **Resource Availability:**
 - The project has an experienced team and access to necessary tools.

2.11 Homogenization process

The homogenization process for CONNECT-KITCHEN involves ensuring consistency across different platform components, including:

Consistent User Interface:

The design will follow a simple, responsive structure with React to ensure responsiveness .

Standardized Data Handling:

The platform will use a consistent format for data (JSON) exchanged between the frontend and backend via RESTful APIs.

Unified Database Schema:

The database structure will be optimized for performance, ensuring data consistency and efficient query handling.

2.12 Stakeholders

- **General Users:** Explore, share, and manage recipes; interact socially through likes, comments, and follows.
- **Content Creators:** Regular contributors (e.g., food bloggers) who drive engagement.
- **Admins:** Manage users and moderate content via the admin panel.
- **Developers:** Build and maintain the platform using React, Node.js, and Express.
- **Database Admins:** Handle database design, performance, and data integrity.
- **UI/UX Designers:** Ensure the app is user-friendly and visually appealing.
- **Project Manager:** Coordinates development, gathers requirements, and tracks progress.

2.13 Use Case Description

USER/CUSTOMER Use Case :

Title: User Login

Description: A user can log into the platform using their email and password. Once logged in, they can access their dashboard, view their profile, browse for recipes, can perform social media functionalities and can place orders.

Preconditions: User must have a registered account, if not then sign up .

Postconditions: Users are successfully logged in and redirected to their dashboard.

Functional Requirements:

- The system must validate email and password input fields for correctness.
- The system must hash passwords before storing them in the database for maintaining confidentiality and integrity .

Admin Use Case :

Title: Admin supervision

Description: An admin can delete inappropriate files, post deletions, can generate sales reports and will manage comment sections to keep a healthy environment. They can also review order history.

Preconditions: Admin can only login with admin domain credentials.

Postconditions: database should be updated successfully.

Functional Requirements:

- The system must allow admin to look through users' profiles.
- The system must allow admin to look into order history.

3. External Interface Requirements

3.1 User Interface

The application provides the following interfaces for end-users:

1. Customer Interface:

- **Login/Register Page:** Users can sign in, register, or reset their password.
- **Home Page:** Displays featured products and categories, like and comments .
- **Product Detail Page:** Includes product images, ingredients , pricing , and comments.
- **Checkout Page:** Captures user address, payment information, and confirms orders.

2. Admin Interface:

- **Dashboard:** Overview of sales and orders.
- **Product Management:** CRUD operations for product categories.

3.2 Hardware Interfaces

1. Supported Device Types

- **User Devices:** Desktop (PCs and laptops,) that can access the website for user registration, login, viewing profiles, creating posts, commenting, liking, and ordering.
- **Admin Devices:** Desktop or mobile devices used by administrators to log in, review order history, and allocate discounts.
- **Server:** A server that hosts the website, database, and handles user authentication, data management, and e-commerce functionalities.

2. Logical Characteristics

- The software allows users to create and manage profiles, posts, and interactions (like, comment, follow), which are stored and retrieved from the database.
- Users can browse recipes, place orders, and generate bills, with order information stored in the database for tracking and updating.
- Admins can review order history and assign discount vouchers based on purchase amounts.

3. Physical Characteristics

- **Web Server:** The website is hosted on a physical server with adequate resources (CPU, RAM, storage) to handle user traffic and database operations.
- **Database Server:** A server dedicated to managing the database that stores user credentials, posts, and order data.
- **User Devices:** Devices with internet connectivity to access the platform through browsers..

3.3 Software Interfaces

The e-commerce platform integrates with several software systems:

1. **Frontend Framework:**
 - React.js for the user interface, bundled with Webpack.
2. **Backend API:**
 - API's are developed using Node.js and Express.js.
3. **Database:**

- PostgreSQL as the primary relational database for storing product, user, and order data.
4. **Third-Party Services:**
- **Cloudinary** for image hosting and optimization.
 - **NodeMailer** for sending notifications (SMS/Email).
5. **Operating System Compatibility:**
- Compatible with modern browsers on Windows, macOS, iOS, and Android.

3.4 Software Interfaces

Data Interactions

- **User Data:** User profiles (name, profile picture, followers, following, posts) are stored and retrieved from the database.
- **Post Data:** Recipe names, ingredients, instructions, images, likes, and comments are stored in the database. When users interact (like, comment, follow), this data is updated in real-time.
- **Order Data:** Orders placed by users (including recipes and prices) are stored in the database, and order history is updated upon checkout. Bills are generated based on the order data.

Communication Protocols

- **HTTP/HTTPS:** Used for communication between user devices and the server via web requests for actions like logging in, creating posts, viewing profiles, browsing recipes, and placing orders.
- **Database Protocols:** SQL-based protocols (, PostgreSQL) for querying and managing user data, posts, and orders.
- **WebSocket/REST APIs:** For real-time updates on posts (likes, comments) and order status

4. System Features

4.1 Admin Features

- View sales reports.
- Easily sort and manage categories, and orders.
- Add or update categories seamlessly.
- View customer profiles.
- Issue voucher.

4.2 User Features

- Quickly search for products and categories.
- Add items to the cart.
- Apply Coupon to your cart and avail discounts.
- Upload post
- Like, comment and follow others.

5. Non-Functional Requirements

The system must meet the following non-functional requirements to ensure reliability, performance, and security:

- Security: All user data should be encrypted in transit and at rest. User passwords should be hashed using industry-standard algorithms.
- Availability: The system should have 99.9% uptime.
- Usability: The user interface should be easy to navigate, with minimal learning curve for both admins and customers.
- Scalability: The system should support future growth in terms of users, products, and orders.

5.1 Performance Requirements

- **Login Response Time:** Login should occur within 2 seconds.
- **Post Creation:** Posting a recipe should take no longer than 5 seconds.
- **Search & Filters:** Search results should be returned within 2 seconds, even with up to 1000 posts.
- **Order Checkout:** Checkout process and bill generation should complete within 5 seconds.
- **Scalability:** System should support up to 10,000 concurrent users without performance degradation.

5.2 Safety Requirements

Data Loss Prevention: Daily backups of the database with offsite storage.

Data Encryption: Encrypt personal data (e.g. password) during storage and transmission.

5.3 Software Quality Attributes

Usability: System should be user-friendly with a minimal learning curve.

Reliability: Ensure 99.9% uptime for the platform.

Scalability: Must handle up to 10,000 users concurrently without performance issues.

Maintainability: Code should be modular, with clear documentation to ease updates and fixes.

5.4 Business Rules

User Roles: Users can create posts, interact with others, and place orders; admins can review orders and assign discounts.

Discount Allocation: Admins can issue discount vouchers for orders over 5000 .

6.Conclusion

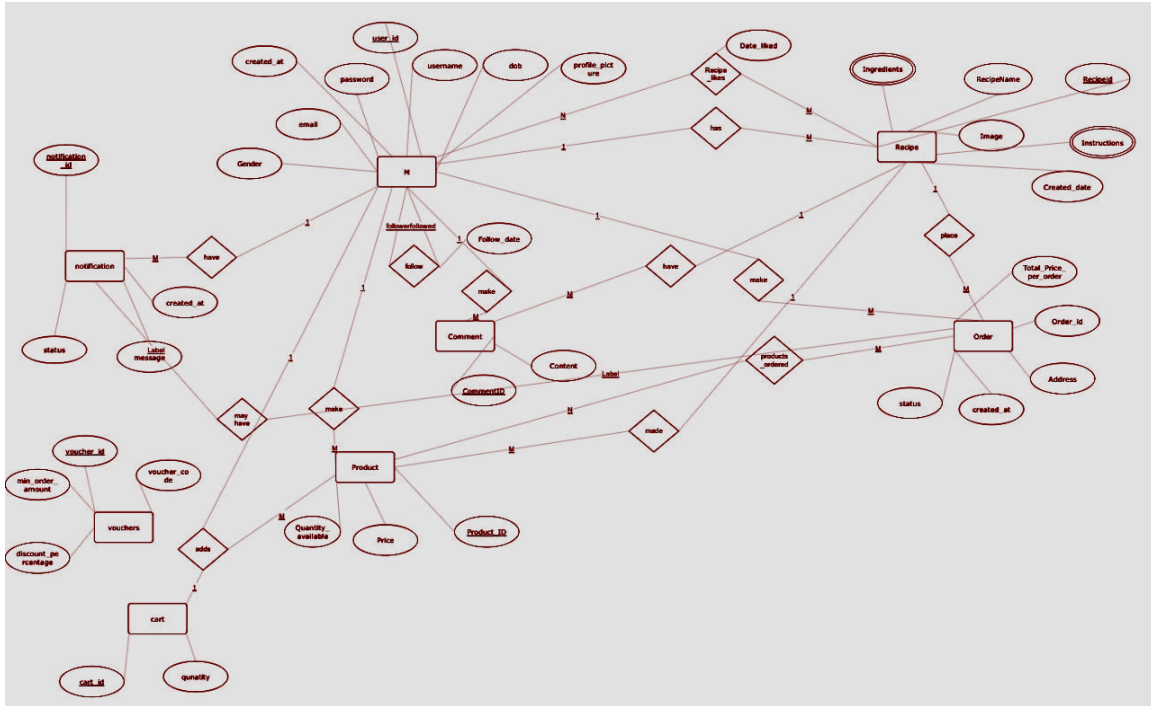
The Software Requirements Specification for **Connect Kitchen** provides a comprehensive overview of the system's purpose, functionality, and technical requirements. By addressing both functional and non-functional aspects, this document serves as a blueprint for the development team and a reference for stakeholders.

The outlined requirements ensure that **Connect Kitchen** will deliver a seamless shopping and social media experience for users and an efficient management platform for administrators. The integration of modern technologies, scalability, and secure practices ensures the system will meet current demands and adapt to future growth.

By following this SRS, the development team aims to deliver a high-quality e-commerce platform that exceeds expectations, ensuring a successful launch and ongoing support for **Connect Kitchen**.

Appendix B: Analysis Models

Entity Relationship Diagram :



Sequence Activity systems architecture :

- **Three-Tier Architecture:**

A three-tier architecture divides the system into three layers: presentation, logic, and data.

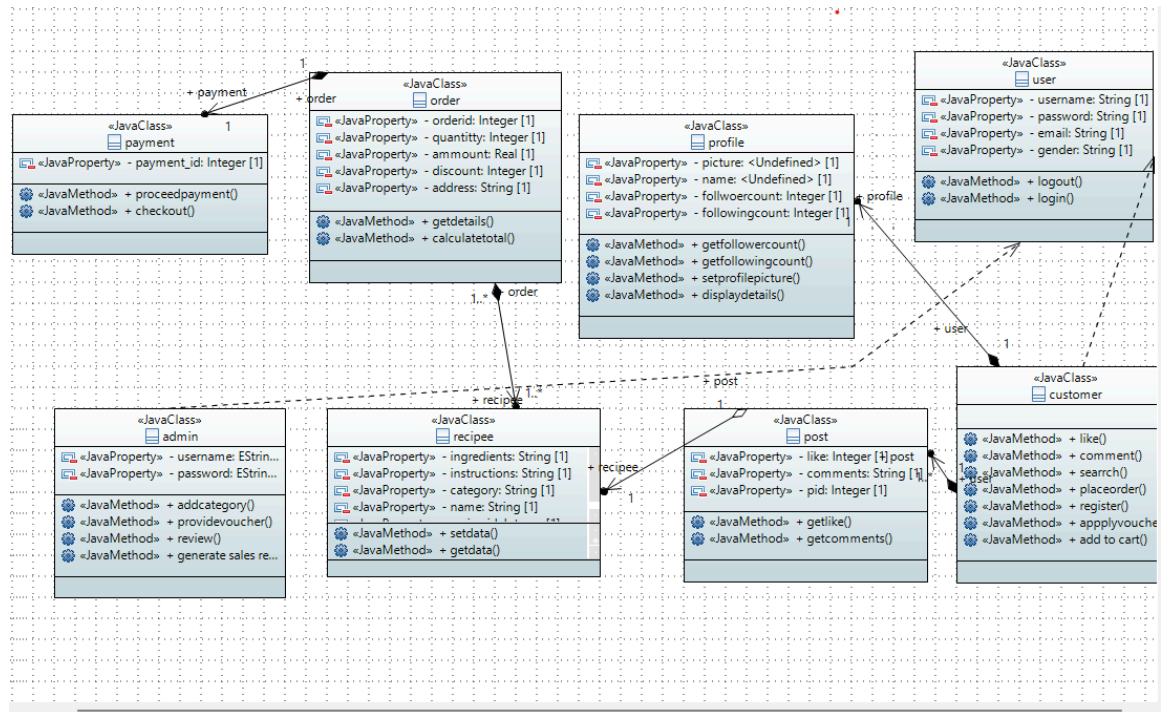
How it Applies to Connect Kitchen:

- **Presentation Tier:** The **React application** that serves the user interface, displaying products, profile, and other UI components to the user.
- **Logic Tier:** The **API layer** (Express.js or Node.js back-end), which handles the business logic (user authentication, cart management, order processing, etc.).
- **Data Tier:** The **database** (PostgreSQL) that stores and retrieves data like user information, product details, and order histories.

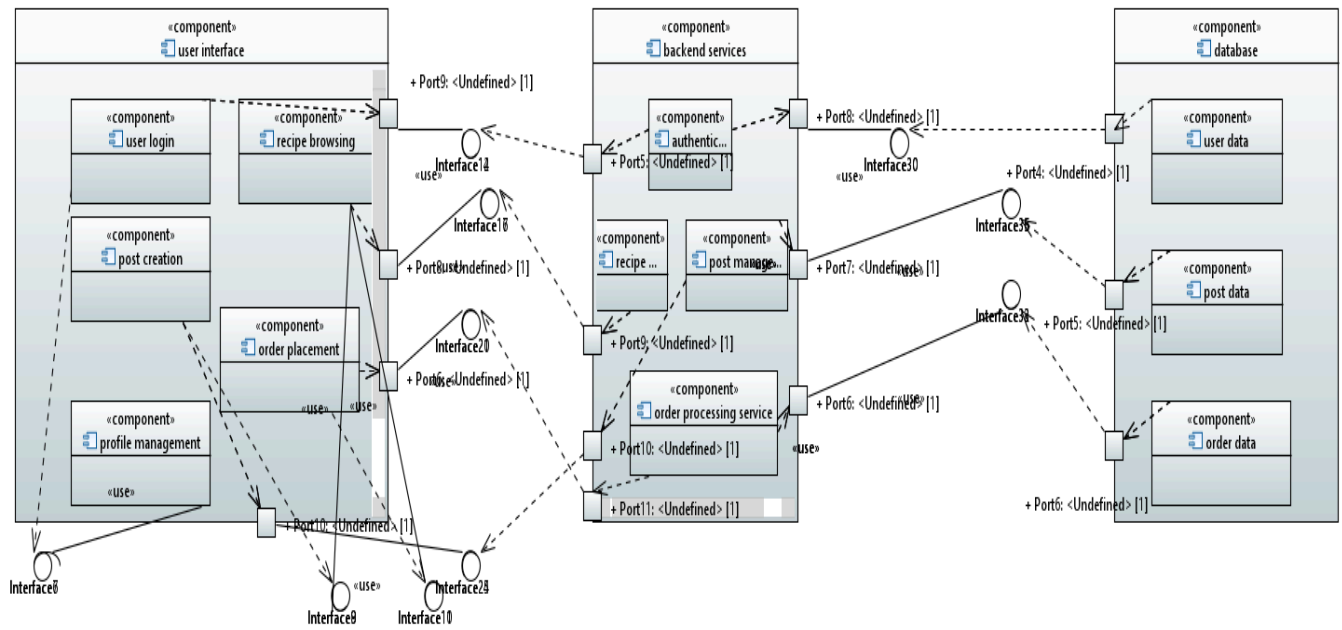
Flow:

1. The **client** (React app) sends requests to the **logic tier** (API).
2. The **logic tier** processes the request and communicates with the **data tier** (database) to retrieve or store information.
3. The **logic tier** sends back the processed data to the **presentation tier** (React app) for the user to interact with.

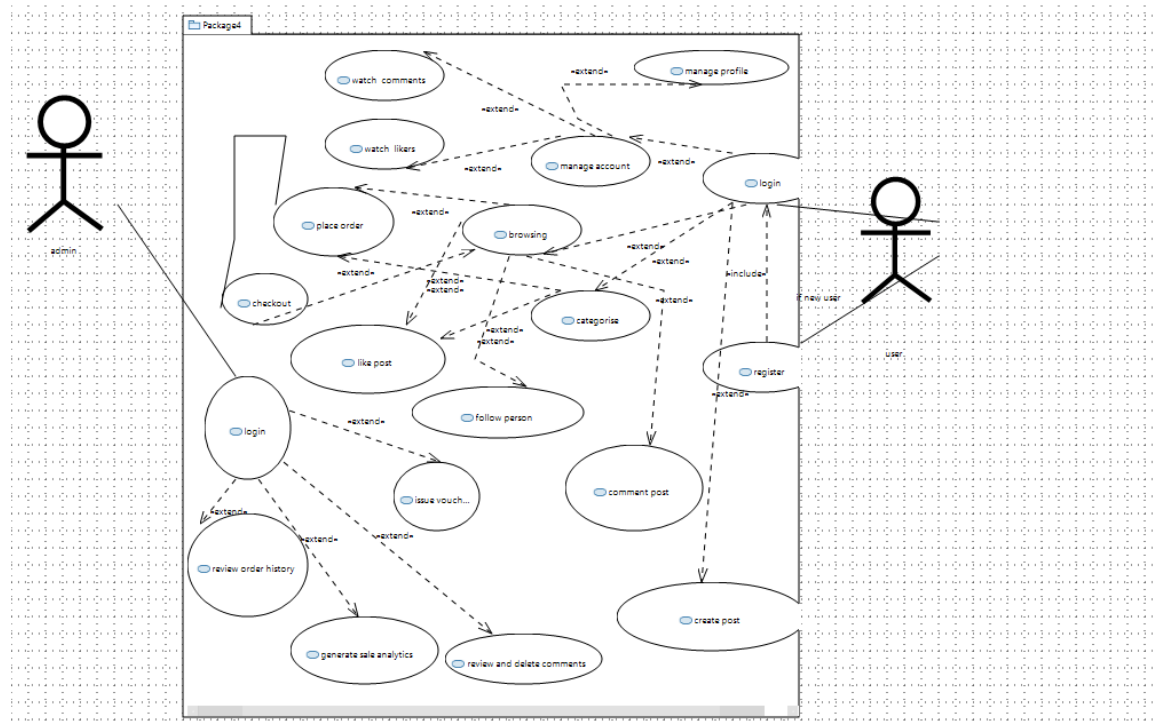
Class Diagram :



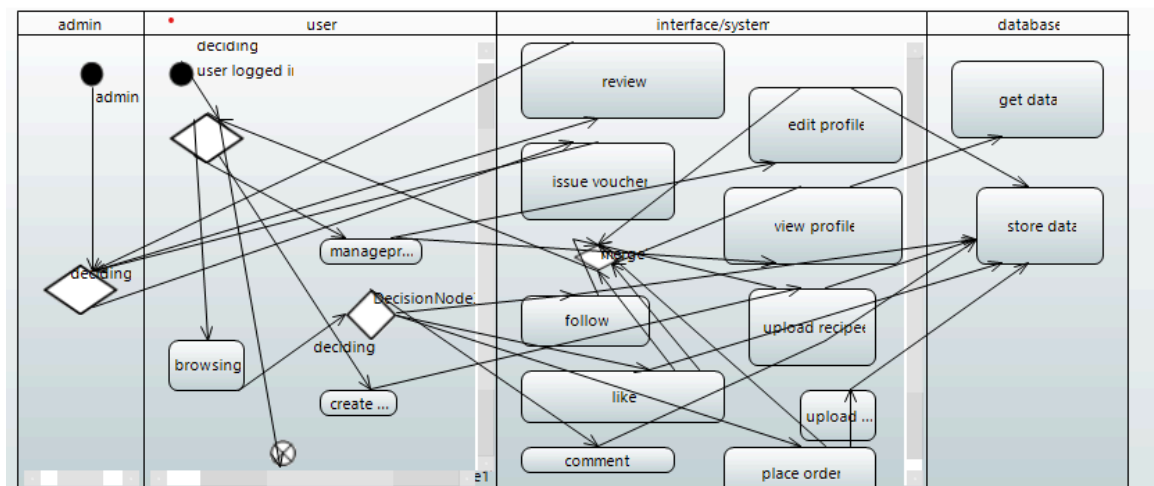
Component Diagram:



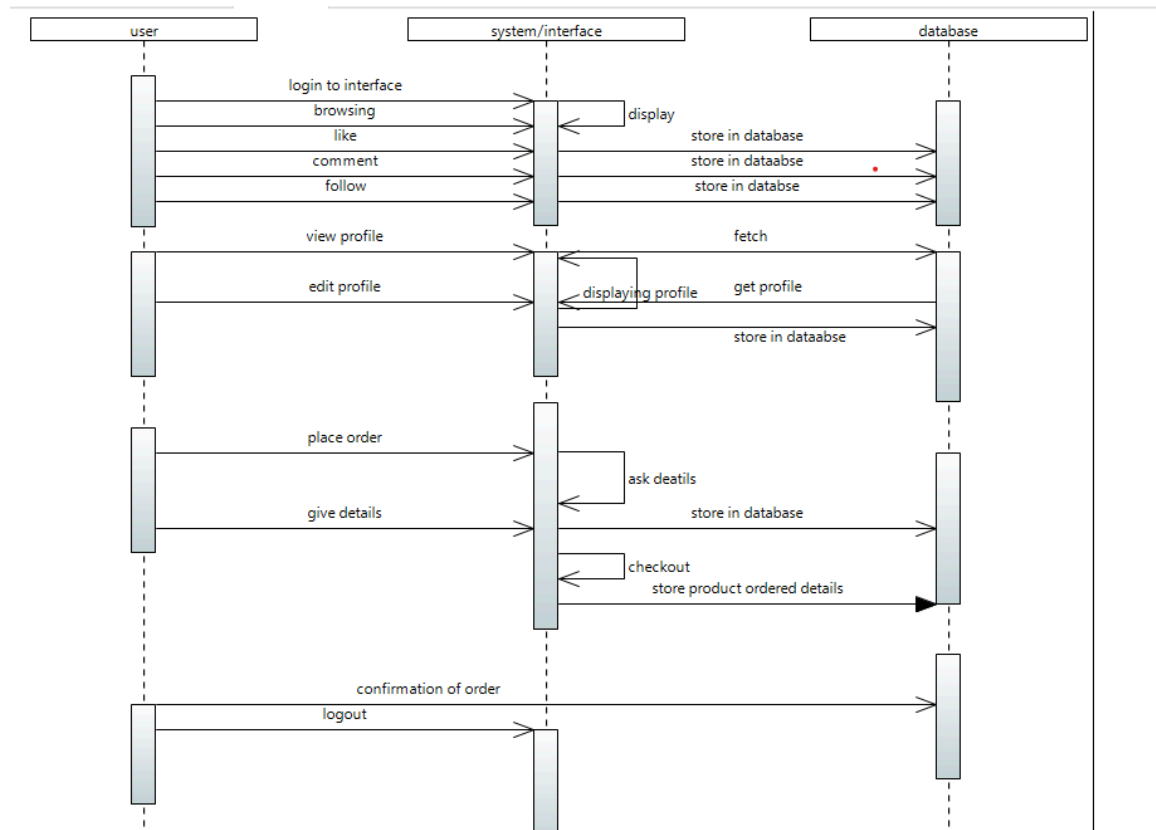
Use Case diagram:



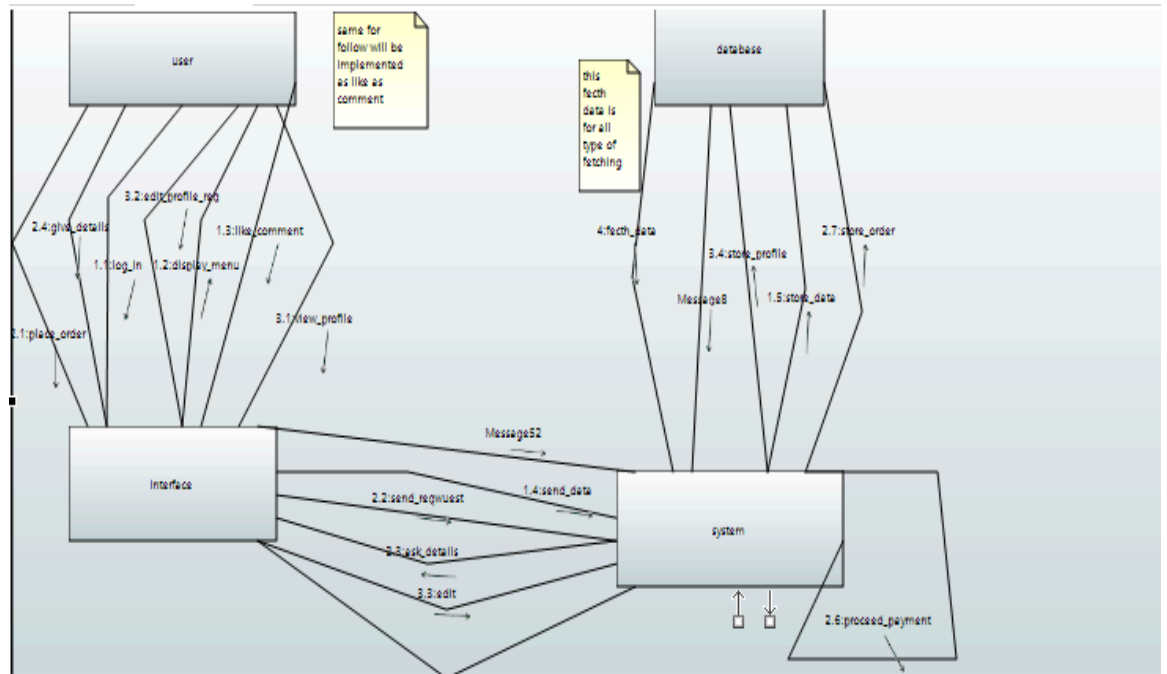
Activity diagram :



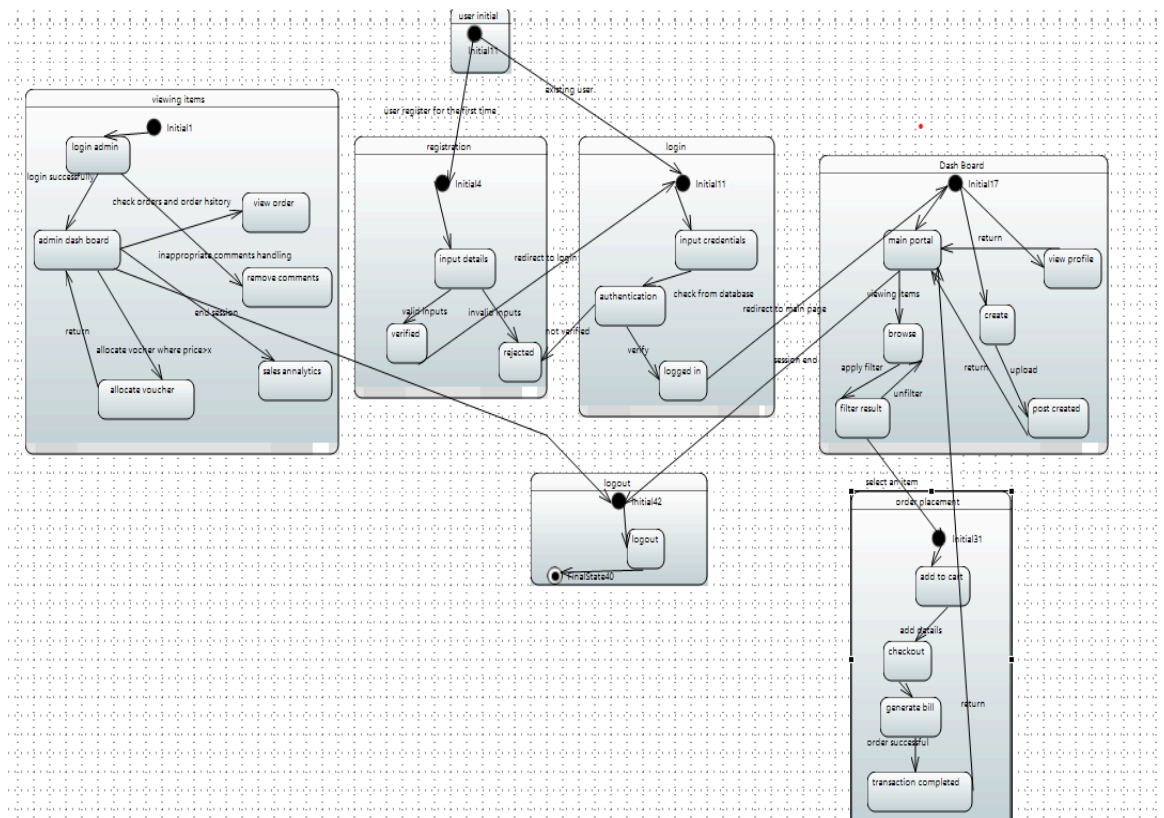
Sequence diagram:



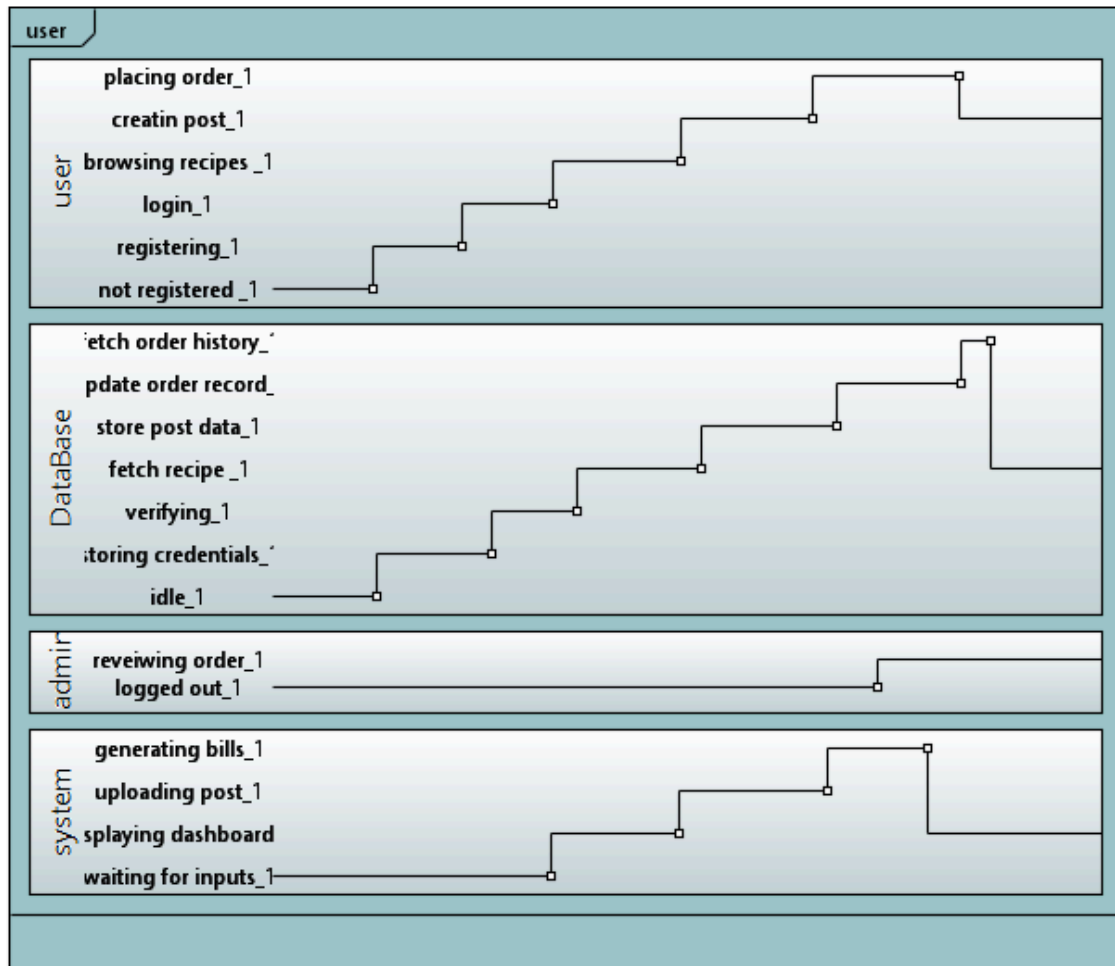
Collaboration Diagram:



State Chart:



Timing diagram:



Deployment diagram

